MATERIA

INF761

Desarrollo de Videojuegos

Guía de Laboratorio 3

"Flappy Clone": Scripting y Ciclo de Vida de MonoBehaviour

Carrera: Ingeniería Informática
Docente: M. Sc. Huáscar Fedor Gonzales Guzmán
Semestre: I-2026
Universidad: Universidad Autónoma Tomás Frías (UATF)

1. Objetivos

Objetivo General

Desarrollar un clon funcional del videojuego Flappy Bird aplicando el ciclo de vida de MonoBehaviour, generación procedural de obstáculos mediante prefabs y spawners, sistemas de puntuación con triggers y gestión de escenas en Unity.

Objetivos Específicos

- Identificar y aplicar correctamente los métodos del ciclo de vida de MonoBehaviour (Awake, Start, Update, FixedUpdate, OnEnable, OnDisable, OnDestroy).
- Implementar movimiento físico del pájaro con Rigidbody2D y AddForce(ForceMode2D.Impulse).
- Crear y gestionar Prefabs para los obstáculos (tubos), generándolos proceduralmente con InvokeRepeating.
- Construir un sistema de puntuación con OnTriggerEnter2D y actualizar la UI TextMeshPro en tiempo real.
- Configurar correctamente Tags, Layers, Colliders y el Canvas de Unity para integrar todos los sistemas.
- Detectar Game Over, mostrar panel de fin de juego y reiniciar la escena con SceneManager.LoadScene.

2. Requisitos Previos

CAMPO / PROPIEDAD	VALOR A CONFIGURAR
Unity LTS 2022.3+	Con módulo 2D Sprite y TextMeshPro instalados desde Package Manager.
Conocimiento previo	GameObjects, componentes, física 2D, colisiones (Guías 1 y 2).
Assets de sprites	Pájaro (1–2 frames), 2 sprites de tubo (top/bottom), sprite de fondo.
C# básico	Variables, funciones, condicionales, clases, herencia simple.
Visual Studio / Rider	Integrado con Unity (instalar desde Unity Hub → Módulos).

3. Marco Teórico

3.1 Ciclo de vida de MonoBehaviour

En Unity, todo componente de scripting hereda de MonoBehaviour. Esta clase base expone métodos especiales (event functions) que el motor invoca automáticamente en momentos concretos de la vida

de un GameObject. Comprender el orden exacto de ejecución previene errores de inicialización, referencias nulas y comportamientos erráticos.

CICLO DE VIDA DE MONOBEHAVIOUR	
Awake() Inicialización	Primera llamada al instanciar el objeto. Úsalo para inicializar referencias internas con GetComponent<>(). Se ejecuta incluso si el componente está desactivado.
OnEnable() Activación	Se llama cada vez que el objeto o componente se activa (SetActive(true)). Útil para reiniciar estado en el Object Pooling.
Start() Inicio único	Ejecutado UNA SOLA VEZ, antes del primer frame, DESPUÉS de que todos los Awake() hayan corrido. Ideal para referencias cruzadas a otros objetos.
FixedUpdate() Física (50 Hz)	Se ejecuta a intervalo fijo (0.02 s = 50 veces/segundo). INDEPENDIENTE de los FPS. Aquí va toda la lógica física: Rigidbody, AddForce, velocidades.
Update() Por frame	Se ejecuta UNA VEZ POR FRAME. La frecuencia varía con los FPS. Aquí va: lectura de Input, lógica de juego, animaciones no físicas.
LateUpdate() Post-frame	Después de todos los Update() del frame. Ideal para cámaras que siguen objetos: el objeto ya terminó de moverse antes de que la cámara actualice.
OnDisable() Desactivación	Cuando el objeto o componente se desactiva. Cancelar InvokeRepeating, desuscribirse de eventos, limpiar referencias temporales.
OnDestroy() Destrucción	Cuando el objeto es destruido con Destroy(). Liberar recursos, guardar datos de partida.

`Flujo: Awake → OnEnable → Start → [ FixedUpdate → Physics → Update → LateUpdate ]* → OnDisable → OnDestroy`

AWAKE vs START — regla práctica

Awake(): auto-inicialización interna. Obtén referencias de tu propio objeto aquí (GetComponent). Se ejecuta aunque el componente esté desactivado.

Start(): inicialización cruzada. Accede a otros objetos de la escena aquí; todos sus Awake() ya terminaron.

Nunca hagas GetComponent<>() en Update() — es costoso. Hazlo en Awake() y guarda la referencia en una variable privada.

UPDATE vs FIXEDUPDATE — diferencia crítica

Update(): corre una vez por frame. A 60 FPS = 60 veces/seg, a 30 FPS = 30 veces/seg. NUNCA apliques fuerzas físicas aquí.

FixedUpdate(): corre a 50 Hz (0.02 s fijo) independientemente de los FPS. TODA la lógica con Rigidbody2D va aquí.

Movimiento no físico frame-rate independiente: multiplica por Time.deltaTime en Update().

3.2 Prefabs

Un Prefab es una plantilla reutilizable de un GameObject con todos sus componentes y valores. Se instancia con Instantiate() en tiempo de ejecución. Modificar el Prefab en el Editor actualiza todas las instancias automáticamente (excepto propiedades sobreescritas por override). Los tubos del juego serán Prefabs instanciados por el Spawner.

3.3 InvokeRepeating — spawn periódico

InvokeRepeating("NombreDelMétodo", tiempoInicial, intervalo) llama a un método cada intervalo segundos sin necesidad de gestionar un contador en Update(). CancelInvoke() detiene todas las invocaciones pendientes. Es más eficiente que un timer manual porque el motor gestiona la planificación.

3.4 Colliders y Triggers

Un Collider2D con Is Trigger = true detecta solapamientos sin generar fuerzas físicas. El método OnTriggerEnter2D(Collider2D other) se invoca cuando otro collider entra en la zona. Para que funcione: al menos uno de los dos objetos debe tener Rigidbody2D, ambos deben tener Collider2D, y el objeto que detecta debe tener Is Trigger = true.

3.5 Object Pooling (concepto introductorio)

Instantiate() y Destroy() son costosos: reservan y liberan memoria generando pausas por el Garbage Collector. El Object Pooling pre-instancia objetos y los activa/desactiva en lugar de crearlos/destruirlos. En esta guía los tubos usan Destroy() para mayor simplicidad; el concepto se practica en el Ejercicio E2.

4. Desarrollo Paso a Paso

1

Crear el proyecto y configurar la cámara

Abre Unity Hub → New Project → selecciona la plantilla '2D (Built-In Render Pipeline)'. Nómbralo 'FlappyClone'. Una vez abierto, selecciona la Main Camera en la Jerarquía y configura en el Inspector:

CAMPO / PROPIEDAD	VALOR A CONFIGURAR
Transform → Position	X: 0 | Y: 0 | Z: -10
Camera → Projection	Orthographic
Camera → Size	5
Camera → Background	Color sólido negro (#1A1A2E) o el color de tu fondo de juego

ORTHOGRAPHIC SIZE = 5

Con Size = 5 la cámara muestra 10 unidades de alto (de -5 a +5 en el eje Y) y ~17.8 unidades de ancho en pantalla 16:9.

Los valores de posición en los scripts (minY, maxY, DestroyX) están calculados para esta configuración.

2 Importar sprites y construir la jerarquía de la escena

Arrastra tus sprites desde el Explorador de archivos a la carpeta Assets/Sprites en el panel Project de Unity. Luego crea la siguiente jerarquía en la ventana Hierarchy (+→ GameObject o clic derecho en la Jerarquía):

```
SampleScene
├── Main Camera                    (ya existe)
├── Background                     (GameObject → 2D Object → Sprite)
├── Bird                           (GameObject → 2D Object → Sprite)
├── Spawner                        (GameObject → Create Empty)
├── GroundCollider                 (GameObject → Create Empty)
├── TopCollider                    (GameObject → Create Empty)
└── Canvas                         (GameObject → UI → Canvas)
    ├── ScoreText                  (clic derecho Canvas → UI → Text-TextMeshPro)
    └── GameOverPanel              (clic derecho Canvas → UI → Panel)
        ├── GameOverText           (clic derecho GameOverPanel → UI → Text-TMP)
        └── RestartButton          (clic derecho GameOverPanel → UI → Button-TMP)
```

INSTALAR TEXTMESHPRO

La primera vez que crees un elemento TextMeshPro, Unity mostrará un popup 'TMP Essentials'.

Haz clic en 'Import TMP Essentials'. Si no aparece: Window → TextMeshPro → Import TMP Essential Resources.

Sin este paso los textos de la UI no renderizarán correctamente.

3 Configurar el GameObject Background

Arrastra tu imagen desde el Finder (Explorador de archivos) directamente a la carpeta Assets en el panel Project de Unity

3a — Configurar la textura

Haz clic en la imagen en el panel Project

CAMPO / PROPIEDAD	VALOR A CONFIGURAR
Sprite Mode	Single
Pixels Per Unit	50
Mesh Type	Full Rect

Wrap Mode	Repeat
Clic en Apply	

3b — Configurar el tiled

CAMPO / PROPIEDAD	VALOR A CONFIGURAR
Sprite	Arrastra el Sprite “background-day”
Draw Mode	Tiled
Mwidth y Height	20 y 10
Order in Layer	-1

4

Configurar el GameObject Bird — Rigidbody2D, Collider y Tag

Selecciona el GameObject Bird en la Jerarquía. Asigna su sprite y añade los componentes físicos desde el Inspector:

4a — Componentes del pájaro:

CAMPO / PROPIEDAD	VALOR A CONFIGURAR
SpriteRenderer → Sprite	Arrastra el sprite del pájaro desde Assets/Sprites
Add Component → Rigidbody2D	Gravity Scale: 2.5 | Freeze Rotation Z: ✓ | Collision Detection: Continuous
Add Component → CircleCollider2D	Ajusta el radio en la vista Scene hasta que cubra el sprite sin sobrepasar
Tag (desplegable arriba del Inspector)	Haz clic → Add Tag → escribe 'Player' → guarda → regresa a Bird y asigna 'Player'

EL TAG 'PLAYER' ES OBLIGATORIO

El script ScoreZone detecta el paso del pájaro con CompareTag("Player").

Si el pájaro no tiene el tag 'Player', el score NUNCA subirá aunque todo lo demás esté correcto.

Proceso: Inspector del Bird → campo Tag (arriba del todo) → Add Tag... → '+' → escribe Player → Save → vuelve a seleccionar Bird → asigna Player en el desplegable Tag.

5

Configurar los colliders de suelo y techo (Game Over por bordes)

Estos dos GameObjects vacíos actúan como paredes invisibles. Selecciona GroundCollider y configura:

5a — GroundCollider (suelo):

CAMPO / PROPIEDAD	VALOR A CONFIGURAR
Transform → Position	X: 0 | Y: -6 | Z: 0
Add Component → BoxCollider2D	Size: X: 20 | Y: 1 (barra horizontal ancha)
BoxCollider2D → Is Trigger	☐ DESACTIVADO — colisión sólida para OnCollisionEnter2D
Tag	Haz clic en Add Tag → escribe 'Ground' → guarda → asigna 'Ground' a GroundCollider

5b — TopCollider (techo):

CAMPO / PROPIEDAD	VALOR A CONFIGURAR
Transform → Position	X: 0 | Y: 6 | Z: 0
Add Component → BoxCollider2D	Size: X: 20 | Y: 1
Tag	Asigna el mismo tag 'Ground' (ya lo creaste en el paso anterior)

POR QUÉ USAR TAG 'GROUND' PARA AMBOS

El script BirdController detecta colisión con tag 'Pipe' O tag 'Ground'. Reutilizar el tag Ground para el techo simplifica el código sin perder claridad.

Alternativamente puedes crear el tag 'Ceiling' y añadirlo a la condición en OnCollisionEnter2D.

6

Script BirdController.cs — movimiento, salto y detección de Game Over

Crea la carpeta Assets/Scripts. Dentro: clic derecho → Create → C# Script → nombra 'BirdController'. Abre el archivo, borra todo y escribe:

```
using UnityEngine;

public class BirdController : MonoBehaviour
{
    [Header("Movimiento")]
    [SerializeField] private float jumpForce = 6f;
```

```
    [Header("Inclinacion visual")]
    [SerializeField] private float rotationUp    =  30f;
    [SerializeField] private float rotationDown  = -90f;
    [SerializeField] private float rotationSpeed =   5f;

    private Rigidbody2D rb;
    private bool isAlive = true;

    // Awake: cache de referencias propias. Se ejecuta al instanciar.
    private void Awake()
    {
        // GetComponent busca el Rigidbody2D en este mismo GameObject
        rb = GetComponent<Rigidbody2D>();
    }

    // Start: una sola vez antes del primer frame
    private void Start()
    {
        isAlive = true;
    }

    // Update: lectura de input — una vez por frame
    private void Update()
    {
        if (!isAlive) return;

        // Saltar con Espacio o clic izquierdo
        if (Input.GetKeyDown(KeyCode.Space) || Input.GetMouseButtonDown(0))
            Jump();

        // Inclinacion visual segun velocidad vertical
        float targetAngle = rb.velocitrb.linearVelocity.y > 0 ? rotationUp : rotationDown;
        float smoothAngle = Mathf.LerpAngle(
            transform.eulerAngles.z,
            targetAngle,
            Time.deltaTime * rotationSpeed
        );
        transform.rotation = Quaternion.Euler(0f, 0f, smoothAngle);
    }

    private void Jump()
    {
        // Resetear velocidad Y para que cada salto sea consistente
        rb.linearVelocity = new Vector2(rb.linearVelocity.x, 0f);
        // AddForce con Impulse aplica la fuerza en un solo frame
        rb.AddForce(Vector2.up * jumpForce, ForceMode2D.Impulse);
    }

    // OnCollisionEnter2D: colision fisica con tubo o suelo/techo
    private void OnCollisionEnter2D(Collision2D col)
```

```
    {
        if (col.gameObject.CompareTag("Pipe") ||
            col.gameObject.CompareTag("Ground"))
        {
            Die();
        }
    }

    public void Die()
    {
        if (!isAlive) return;   // evitar llamadas duplicadas
        isAlive = false;
        rb.simulated = false;   // detener toda la fisica del pajaro
        GameManager.Instance.GameOver();
    }

    // OnDisable: limpiar estado al desactivar (usado por Object Pooling)
    private void OnDisable()
    {
        rb.linearVelocity = Vector2.zero;
    }
}
```

6a — Asignar el script al pájaro:

Selecciona el GameObject Bird en la Jerarquía → arrastra el archivo BirdController.cs desde Assets/Scripts al Inspector del Bird. O usa Add Component → busca 'BirdController'.

7

Crear el Prefab de tubo con ScoreTrigger y tag 'Pipe'

El tubo es el obstáculo principal. Tiene tres partes: tubo superior, tubo inferior, y un trigger invisible para el score.

7a — Construir la estructura del tubo en la escena:

```
Pipe                        (GameObject → Create Empty)
├── PipeTop                 (clic derecho Pipe → 2D Object → Sprite)
├── PipeBottom              (clic derecho Pipe → 2D Object → Sprite)
└── ScoreTrigger            (clic derecho Pipe → Create Empty)
```

7b — Configurar cada elemento:

CAMPO / PROPIEDAD	VALOR A CONFIGURAR
PipeTop → SpriteRenderer	Asigna el sprite “pipe-green”.
PipeTop → Transform → Position	Y: 4

PipeTop → Transform → Scale	Y: -1
PipeTop → BoxCollider2D	Ajustado al sprite del tubo (Is Trigger: ☐ desactivado)
PipeTop → Tag	Add Tag → 'Pipe' → guarda → asigna 'Pipe' a PipeTop
PipeBottom → SpriteRenderer	Mismo Sprite.
PipeBottom → Transform → Position	Y: -4
PipeTop → Transform → Scale	Y: 1
PipeBottom → BoxCollider2D	Ajustado al sprite (Is Trigger: ☐ desactivado). Asigna tag 'Pipe'.
ScoreTrigger → BoxCollider2D	Size: X: 0.3 | Y: 1.5 (el hueco entre tubos). Is Trigger: ✓ ACTIVADO
ScoreTrigger → Transform → Position	Y: 0 (centro del hueco)

7c — Convertir en Prefab:

Crea la carpeta Assets/Prefabs. Arrastra el GameObject Pipe desde la Jerarquía hacia esa carpeta en el panel Project. El nombre 'Pipe' en la Jerarquía cambiará a color azul confirmando que es una instancia de Prefab. Ahora puedes borrar esa instancia de la escena (ya no la necesitas ahí).

8

Script PipeMover.cs — movimiento continuo del tubo

Crea Assets/Scripts/PipeMover.cs. Este script mueve el tubo hacia la izquierda y lo destruye al salir de pantalla:

```
using UnityEngine;

public class PipeMover : MonoBehaviour
{
    // HideInInspector: no visible en el Inspector pero accesible por código
    [HideInInspector] public float speed = 4f;

    // Posicion X en que el tubo sale completamente de la pantalla
    private const float DestroyX = -12f;

    // FixedUpdate: movimiento fisico a 50 Hz, independiente de FPS
    private void FixedUpdate()
    {
        // Mover hacia la izquierda multiplicando por Time.fixedDeltaTime
```

```
        // para que la velocidad sea en unidades/segundo (no unidades/frame)
        transform.position += Vector3.left * speed * Time.fixedDeltaTime;

        if (transform.position.x < DestroyX)
            Destroy(gameObject);    // OnDestroy se llamara aqui
    }
}
```

¿POR QUÉ NO usar transform.Translate en Update()?

transform.Translate en Update() produce movimiento que varía con los FPS (a 30 FPS el tubo avanza el doble que a 60 FPS si no multiplicas por Time.deltaTime).

Al usar FixedUpdate() con Time.fixedDeltaTime la física del tubo es idéntica en cualquier máquina.

7a — Aplicar al Prefab:

Haz doble clic en el Prefab Pipe en Assets/Prefabs para abrirlo en modo Prefab Edit. Selecciona el GameObject raíz Pipe → Add Component → PipeMover. Guarda con el botón 'Save' arriba de la vista Scene. Sal del modo Prefab con la flecha ← junto al nombre del Prefab.

9

Script PipeSpawner.cs — generación procedural de tubos

Crea Assets/Scripts/PipeSpawner.cs. Asígnalo al GameObject Spawner en la escena:

```
using UnityEngine;

public class PipeSpawner : MonoBehaviour
{
    [Header("Prefab y configuracion")]
    [SerializeField] private GameObject pipePrefab;
    [SerializeField] private float spawnInterval = 2.2f;
    [SerializeField] private float pipeSpeed     = 4f;

    [Header("Rango vertical del hueco (camara size=5)")]
    [SerializeField] private float minY = -1.8f;
    [SerializeField] private float maxY =  1.8f;

    private void Start()
    {
        // Parametros: nombre del método, retardo inicial, intervalo
        // nameof() es más seguro que un string literal — el compilador
        // detecta si el nombre del método cambia
        InvokeRepeating(nameof(SpawnPipe), 1f, spawnInterval);
    }

    private void SpawnPipe()
    {
        float randomY  = Random.Range(minY, maxY);
```

```
        Vector3 spawnPos = new Vector3(8f, randomY, 0f);

        // Instantiate crea una copia del Prefab en la posicion dada
        GameObject pipe = Instantiate(pipePrefab, spawnPos, Quaternion.identity);

        // Configurar la velocidad del tubo recien creado
        pipe.GetComponent<PipeMover>().speed = pipeSpeed;
    }

    public void StopSpawning()
    {
        CancelInvoke(nameof(SpawnPipe));
    }

    // OnDisable cancela invocaciones al desactivar el objeto
    private void OnDisable()
    {
        CancelInvoke();
    }
}
```

9a — Configurar el Spawner en el Inspector:

CAMPO / PROPIEDAD	VALOR A CONFIGURAR
Selecciona el GameObject Spawner	En la Jerarquía → Inspector → Add Component → PipeSpawner
PipeSpawner → Pipe Prefab	Arrastra el Prefab 'Pipe' desde Assets/Prefabs al campo 'Pipe Prefab'
Spawn Interval	2.2 (segundos entre tubos)
Pipe Speed	4 (unidades por segundo)
Min Y / Max Y	-1.8 / 1.8 (rango de posición vertical del hueco)

10

Script ScoreZone.cs — detectar el paso del pájaro

Crea Assets/Scripts/ScoreZone.cs. Debe asignarse al GameObject ScoreTrigger dentro del Prefab:

```
using UnityEngine;

// RequireComponent garantiza que siempre haya un Collider2D en este objeto
[RequireComponent(typeof(Collider2D))]
public class ScoreZone : MonoBehaviour
{
    // OnTriggerEnter2D: se llama cuando otro Collider2D entra en el trigger
    // REQUISITO: este objeto debe tener Is Trigger = true en su Collider2D
```

```
    // REQUISITO: el pájaro debe tener tag "Player"
    private void OnTriggerEnter2D(Collider2D other)
    {
        if (other.CompareTag("Player"))
        {
            GameManager.Instance.AddScore();
        }
    }
}
```

10a — Aplicar ScoreZone al Prefab:

Abre el Prefab Pipe en modo edición (doble clic en Assets/Prefabs/Pipe). Selecciona el hijo ScoreTrigger → Add Component → ScoreZone. Verifica que ScoreTrigger tenga BoxCollider2D con Is Trigger = ✓ activado. Guarda el Prefab.

11

Script GameManager.cs — singleton de score, Game Over y reinicio

Crea Assets/Scripts/GameManager.cs. Crea también un GameObject vacío en la escena llamado 'GameManager' y asígna el script:

```
using UnityEngine;
using UnityEngine.SceneManagement;
using TMPro;

public class GameManager : MonoBehaviour
{
    // Singleton: referencia global unica, accesible con GameManager.Instance
    public static GameManager Instance { get; private set; }

    [Header("Referencias UI")]
    [SerializeField] private TextMeshProUGUI scoreText;
    [SerializeField] private GameObject      gameOverPanel;

    [Header("Referencia al Spawner")]
    [SerializeField] private PipeSpawner pipeSpawner;

    private int  score    = 0;
    private bool gameOver = false;

    private void Awake()
    {
        // Patron Singleton: destruir duplicados
        if (Instance != null && Instance != this)
        {
            Destroy(gameObject);
            return;
        }
        Instance = this;
```

```
    }

    private void Start()
    {
        gameOverPanel.SetActive(false); // ocultar panel al inicio
        UpdateScoreUI();
    }

    public void AddScore()
    {
        if (gameOver) return;
        score++;
        UpdateScoreUI();
    }

    private void UpdateScoreUI()
    {
        scoreText.text = score.ToString();
    }

    public void GameOver()
    {
        if (gameOver) return;
        gameOver = true;
        pipeSpawner.StopSpawning();
        gameOverPanel.SetActive(true); // mostrar panel Game Over
    }

    // Este metodo sera llamado por el boton Restart en el Canvas
    public void RestartGame()
    {
        // Recarga la escena activa desde cero
        SceneManager.LoadScene(SceneManager.GetActiveScene().name);
    }
}
```

10a — Conectar referencias en el Inspector:

Selecciona el GameObject 'GameManager' en la Jerarquía. En el Inspector verás los campos SerializeField. Arrastra:

CAMPO / PROPIEDAD	VALOR A CONFIGURAR
Score Text	Arrastra el GameObject ScoreText (dentro del Canvas) al campo 'Score Text'
Game Over Panel	Arrastra el GameObject GameOverPanel al campo 'Game Over Panel'
Pipe Spawner	Arrastra el GameObject Spawner al campo 'Pipe Spawner'

12

Configurar el Canvas y conectar el botón Restart

La UI necesita configuración explícita en Unity. Este paso conecta el botón de reinicio con el GameManager.

12a — Configurar el Canvas:

CAMPO / PROPIEDAD	VALOR A CONFIGURAR
Canvas → Canvas component	Render Mode: Screen Space - Overlay
Canvas → Canvas Scaler	UI Scale Mode: Scale With Screen Size | Reference: 1920×1080
Canvas → Graphic Raycaster	Dejar valores por defecto

12b — Posicionar ScoreText:

CAMPO / PROPIEDAD	VALOR A CONFIGURAR
ScoreText → Rect Transform	Anchor: Top Center | Pos Y: -60 | Width: 200 | Height: 80
ScoreText → TextMeshProUGUI → Text	0 (texto inicial)
Font Size	72
Alignment	Center
Color	Blanco (#FFFFFF) con sombra si deseas legibilidad

12c — Configurar GameOverPanel:

CAMPO / PROPIEDAD	VALOR A CONFIGURAR
GameOverPanel → Rect Transform	Anchor: Stretch (todo el Canvas).
GameOverText → Text	GAME OVER | Font Size: 80 | Alignment: Center | Width: 300 | Height: 50 | Pos Y: 140
RestartButton → Rect Transform	Anchor: Center | Width: 300 | Height: 100 | Pos Y: -80
RestartButton → Button (TMP) → Text	REINICIAR

12d — Conectar el botón Restart al GameManager:

Este es el paso que más se olvida. Sin él el botón no hace nada al presionarlo:

```
1. Selecciona RestartButton en la Jerarquía
2. En el Inspector → componente Button → sección 'On Click ()'
3. Haz clic en el botón '+' para añadir un listener
4. En el campo vacío (None (Object)) arrastra el GameObject 'GameManager'
5. En el desplegable 'No Function' → selecciona GameManager → RestartGame()
6. El evento debería verse así:
   On Click () ─── 1 listener
   GameManager (GameManager) | GameManager.RestartGame
```

AGREGAR LA ESCENA EN BUILD SETTINGS

SceneManager.LoadScene() solo funciona si la escena está registrada en Build Profiles.

Ve a File → Build Profiles → haz clic en 'Open Scene List'. Deberías ver 'SampleScene' con índice 0.

Sin este paso el juego compilará pero al morir dará un error: 'Scene... couldn't be loaded'.

5. Ejercicios Propuestos

E1 — Dificultad Progresiva

1. Modifica PipeSpawner: cada 5 puntos reduce spawnInterval en 0.15 s (mínimo 0.8 s).
2. Incrementa pipeSpeed en 0.3 por cada 10 puntos (máximo 9 m/s).
3. Tip: usa un evento de C# en GameManager — public event System.Action OnScoreChanged — y suscribe PipeSpawner en su Start().

E2 — Object Pooling para Tubos

1. Integra SimpleObjectPool al PipeSpawner reemplazando Instantiate/Destroy.
2. En SpawnPipe() usa pool.Get() en lugar de Instantiate().
3. Añade un método ReturnToPool() en PipeMover que llame pool.Return(gameObject) en lugar de Destroy().
4. Mide la diferencia en el Profiler de Unity (Window → Analysis → Profiler) entre versión con/sin pool.

E3 — Skins para el Pájaro

1. Añade en BirdController: [SerializeField] private Sprite[] skins;
2. Al presionar 'S' en el menú de inicio, cicla entre skins con un índice.

3 Aplica en Start(): GetComponent<SpriteRenderer>().sprite = skins[selectedIndex].

4 Persiste la selección: PlayerPrefs.SetInt("BirdSkin", index) al cambiar y GetInt en Awake().

7. Bibliografía y Recursos

[1] Unity Technologies. (2024). MonoBehaviour — Unity Scripting Reference. https://docs.unity3d.com/ScriptReference/MonoBehaviour.html

[2] Unity Technologies. (2024). Rigidbody2D.AddForce — Unity Scripting Reference. https://docs.unity3d.com/ScriptReference/Rigidbody2D.AddForce.html

[3] Unity Technologies. (2024). Object Pooling en Unity. https://unity.com/resources/introduction-to-pooling-unity-tutorial-intermediate-gamedev

[4] Unity Technologies. (2024). SceneManager.LoadScene — Unity Scripting Reference. https://docs.unity3d.com/ScriptReference/SceneManagement.SceneManager.LoadScene.html

[5] Unity Technologies. (2024). Tags — Unity Manual. https://docs.unity3d.com/Manual/Tags.html

[6] Halpern, J. (2015). Introduction to Game Design, Prototyping, and Development. Addison-Wesley Professional.

[7] Ferrone, H. (2020). Learning C# by Developing Games with Unity 2020 (5th ed.). Packt Publishing.